Below is a worked example that runs one 500 KB file through Riverbed's design three times: new, unchanged, then with one inserted byte. The unit size (~137 KB), record sizes, ack behaviour and counter steps come from the captures. The rest is my model of what happens inside the boxes, and I've marked it as such, with made-up values where the captures can't show the real ones. These parts are unknown:

- the real segment size and how segment labels look;
- what the counter actually counts (the model below is the hypothesis from last time);
- the number of levels in the hierarchy.

---

0. Setup and assumptions

Item	Value	Status
File	500 KB = 512,000 B, downloaded server → client	example
Sender / receiver	RB-B (server side) / RB-A (client side)	measured roles
Unit size	137,248 B (50 MB / 382)	measured average
Segment size	~128 B average, boundaries chosen by	**assumed** (C3 only shows ≤ ~110 B per insert)

	content	
Hierarchy	unit node → group nodes (~16 segments each) → segments	**assumed**
Node counter N (the 4-byte value in 0x08)	one counter per (box 00224e7f, store 2dfe63xx); starts at 0x2000 in this example	counter scope measured; start value made up
Segment labels	separate namespace from N	**assumed**

	(shown as s…)	
Record ID (0002 xxxx in 0x08, echoed by the 0x09 ack)	its own sequence per connection	measured echo; values made up

Model for N (hypothesis): the 0x08 of each unit carries the **next free node ID**. Both boxes then create that unit's new nodes as N, N+1, …, so the receiver can number them without a list of IDs being sent. The step from one unit's counter to the next is therefore the number of nodes the earlier unit created. This fits the measured steps of 0, +1 and +2.

Cutting the file into units:

Unit	File bytes	Size	Segments (~128 B each)	Groups (~16 segments)
U1	0 – 137,247	137,248	~1,072	~67
U2	137,248 – 274,495	137,248	~1,072	~67
U3	274,496 – 411,743	137,248	~1,072	~67

U4	411,744 – 511,999	100,256	~783	~49
Total		512,000	~4,000	~250

Unit boundaries depend on byte count (measured: 49 MB gives 375 pairs, 50 MB gives 382). Segment boundaries depend on content (assumed). That difference matters in pass 3.

---

1. Pass 1: file is new (cold)

What RB-B does, per unit:

1. Takes 137,248 B from the server stream.
2. Cuts it into ~1,072 segments at content-defined points.
3. Hashes each segment and looks it up: none found, all new.
4. Gives each segment a label in the segment namespace (s0001…s1072 for U1).
5. Builds ~67 group nodes and one unit node. In this model a cold unit creates group nodes in another space, or only the unit node uses N; see the note below the table.
6. Compresses the new bytes with LZ and sends them in **0x00**.
7. Sends **0x08** with record ID, box, store and N.
8. Advances N by the number of nodes created.

Unit	0x08	New	Step to	0x00	0x08	Ack bac

	counterN	nodes created	next	size (random / text)		k
U1	0x2000	1 (unit node 2000)	+1	~140.8 KB / ~46 KB	~150 B, ID 0002 0a01	0x09 echoing 0002 0a01, window 0003fff

						c
U2	0x2001	1	+1	~140.8 KB / ~46 KB	~150 B, 0002 0a02	echo 0a02
U3	0x2002	1	+1	~140.8 KB / ~46 KB	~150 B, 0002 0a03	echo 0a03
U4	0x2003	1	—	~102.9 KB / ~33	~150 B, 0002 0a0	echo 0a04

				KB	4	
After pass 1	N = 0x2004			~525 KB random / ~171 KB text	4 × ~150 B	4 × 14 B

The cold numbers come from measurements: random data costs +2.6% (53.77 MB for 50 MB) and text compresses ~3:1 (17.6 MB).

Note on +1 here: if the group nodes drew from the same counter, a cold unit would jump by ~68

instead of 1. Nothing measured so far shows which is true, because C1's cold phase lost 55% of its packets. A clean cold capture (C2b) answers it:

- steps of 1–2 on new data would confirm that N counts only high-level nodes;
- jumps of ~60–1,000 would mean it also counts groups or segments.

What RB-A does, per unit:

1. Reads the 6-byte header and checks the 2-byte header check.
2. Reads the 0x00, decompresses it and stores the ~1,072 new segments under the labels described there.
3. Reads the 0x08, notes N = 0x2000, and creates unit node 2000 locally.
4. Rebuilds the 137,248 B (here simply the

decompressed bytes) and sends it to the client.

5. Sends a **0x09** echoing the record ID; RB-B can now send more against the 262,140 B window.

Both stores now hold ~4,000 segments, ~250 group nodes and unit nodes 2000–2003.

---

2. Pass 2: same file again (warm)

RB-B, per unit: cuts the unit into the same ~1,072 segments, because content is unchanged. Hashes them: all found. It then creates a new unit node listing the old segments or groups, as the measurements show: sending the same

content again gets new counter values.

Unit	0x08 counterN	New nodes	Step	0x00 contents	0x00 size	Wire for unit
U1	0x2004	1 (new unit node pointing to the sam	+1	compressed list of old labels, no data	~3–8 KB	~6 KB

		e old segments)				
U2	0x2005	1	+1	label list	~3–8 KB	~6 KB
U3	0x2006	1	+1	label list	~3–8 KB	~6 KB
U4	0x2007	1	—	label list	~2–6 KB	~4 KB
After pass 2	N = 0x2008					~22 KB ≈ 4.4

						% of the file

This matches the measured 4–6% warm cost (C1: 3.05 and 2.21 MB per 50 MB; C2: 1.97 MB).

Unit node 2000 already describes exactly the same bytes as U1, yet RB-B sends a new 2004 instead of pointing to 2000. That's why their warm cost doesn't fall on the second, third or fourth repeat (C1 phase 3 still cost 2.2 MB).

RB-A: decompresses the label list, fetches the ~1,072 stored segments, joins them into 137,248 B, sends that to the client, creates node 2004 and acks.

3. Pass 3: one byte inserted at file offset 200,000 (inside U2)

The file is now 512,001 B. Unit boundaries shift by 1 byte from U2 onwards, because units are cut by byte count. Segment boundaries realign right after the edit, because they are content-defined.

Unit	What changed	New segments	New nodes (model)	0x08 N	Step	0x00

U1	nothing	0	1 (unit node)	0x2008	+1	labell list ~5 KB
U2	the segment holding offset 200,000 is now 129	2 small new segments	2 (new group node for the edited group + unit	0x2009	+2	labell list + ~130 B of new bytes, ~5 KB

	B, so its has h is new. Also the last seg men t of U2 is cut 1 byte earli		nod e)				

	er by the shifted unit boundary					
U3	the content is old, but U3 now star	1	2 (group + unit)	0x200b	+2	label list + ~130 B

	ts 1 byte earlier, so its first segment (cut by the unit boundary) is new					

U4	same boundary effect; now 100,257 B	1	2	0x200d	—	label list + ~130 B
After pass 3		4 new segments, ~0.		N = 0x200f		~22.5 KB

		5 KB of data				

So the cost of the edit is about the size of the edited segments, about 0.5 KB. The measured upper bound of ≤110 B per insert in C3 points the same way. The +2 steps after an edit match what C4 mid4k shows (220 of 382 steps were +2). The boundary-shift explanation for the +2 is my model, not something decoded from the bytes.

RB-A: for U2 it takes the ~1,070 old segments from its store, the 2 new ones from the 0x00, and joins them into 137,248 B. It creates nodes 2009 and 200a and acks.

---

4. How the counter grows



```
pass 1 (cold)   U1 2000 → U2 2001 →
U3 2002 → U4 2003      steps +1 +1 +1
(N after = 2004)
pass 2 (warm)   U1 2004 → U2 2005 →
U3 2006 → U4 2007      steps +1 +1 +1
(N after = 2008)
pass 3 (edit)   U1 2008 → U2 2009 → U3
200b → U4 200d      steps +1 +2 +2   (N
after = 200f)
                              ^^^^ edited unit
created 2 nodes
```

The other step values that show up in the

captures:

Step	Meaning in the model	Measured share
0	the earlier unit created no node: fully covered by an existing node (e.g. a repeat inside the same file)	6–18%
+1	earlier unit created only its unit node (unchanged content)	27–85%

+2	earlier unit also created one new group (an edit or a boundary shift)	1–58% (58% in C4 mid4k)
backward / large jump	0x08 points to an old node instead of the next free ID (a reference back to earlier data)	1–4%

The counter is never reset. Pass 2 continues from 2004 and pass 3 from 2008, even on new

TCP connections. This matches C3 (1f67 → 1f69 → …) and C4 (2259 → 225a).

---

5. Summary per pass for the 500 KB file

Pass	Data sent	WAN bytes (random / text)	Counter range
1 cold	all data, compressed	~525 KB / ~171 KB	2000–2003
2	label	~22 KB /	2004–

unchanged	lists only	~22 KB	2007
3 one-byte insert	label lists + ~0.5 KB of new segments	~22.5 KB / ~22.5 KB	2008–200d

For comparison, our NA on the same file (32 chunks of 16 KB):

- Pass 1: 500 KB raw, or ~171 KB with MISSZ.
- Pass 2: 32 HITs, ~0.4 KB today, a few bytes with HITR.
- Pass 3: ~16 KB resent for the edited chunk, plus every later chunk if our chunks are fixed-size and the insert shifts them, or ~0.1 KB with DELTA.

Two measurements would turn the model parts into facts:

- C2b or a clean cold rerun shows whether new data moves the counter by 1 or by hundreds.
- C8c (clear only RB-A's store) shows what happens when a label the sender uses is missing at the receiver.